

COROLLA FIX HELPER

12-Month Strategic Architecture & Product Review

Decision memo and prioritized roadmap

Strategic thesis: deepen before broadening. Make the information -> evidence -> decision -> repair -> record loop exceptional before expanding scope.

Prepared August 2026

1. Executive Summary

The strongest strategic direction for Corolla Fix Helper is not to become a broader AI mechanic, a shop-management system, or a miniature ALLDATA/Mitchell1. It should become exceptionally good at one job:

Turn a personally owned repair-information library into fast, trustworthy, actionable repair guidance - with a visible trail back to the source.

That direction is more differentiated than it may appear. Commercial systems such as ALLDATA, Mitchell1, Snap-on, and garage-management platforms win through breadth: large OEM datasets, wiring information, diagnostics, labor data, estimating, business workflow, and shop operations. Corolla Fix Helper cannot realistically beat those products on breadth, but it does not need to.

Internally, the project has already moved beyond a typical prototype. It has local SQLite and file storage, backup/restore, local OCR, hybrid retrieval, source-page navigation, server-side evidence verification, safety-aware repair planning, browser-history-aware navigation, accessibility work, production smoke tests, and a large automated test suite.

The project has also demonstrated the right engineering instinct: measure before adding sophistication. A proposed PDF reading-order algorithm was rejected after real-corpus testing showed it made actual repair data worse, and a relevance threshold was deliberately left disabled when measurements showed no benefit. That discipline should become the governing product-development rule.

Recommended investment allocation

Investment	Share
Evidence quality, evals, ingestion, retrieval	35%
Core reliability, performance, developer quality	20%
Repair workflow, history, maintenance usefulness	20%
Mobile and hands-busy usability	10%
Diagnostics experiments	5%
Local/offline AI experimentation	5%
New AI modalities / other exploration	5%

Final strategic recommendation: deepen before broadening. Spend the next six months making the information-to-evidence-to-repair loop excellent. Only then expand vehicle scope, diagnostics, local models, or richer multimodal features. **Confidence: Very high.**

2. Current State Assessment

What is already strong

Evidence-backed conclusion: the fundamental architecture fits the product philosophy extremely well. Durable data stays in SQLite and local files, non-AI functionality works without an API key, OCR is local, and external AI is isolated behind injectable boundaries.

The RAG architecture is already modern enough for the current scale: lexical and semantic retrieval are combined, embeddings are stored locally, reranking is optional, and retrieval quality is tested separately from generation. There is no evidence of an architectural crisis that calls for replacement.

Where the project is unusually strong

The strongest component is no longer merely Ask AI. It is the trust architecture around Ask AI: server-owned evidence IDs, quote verification, numeric checks, derived answer statuses, selective citations, and fail-closed rendering when evidence guarantees are not met. The Repair Planner follows a similar model with canonical tasks, trusted server context, completion integrity, evidence verification, safety classification, readiness rules, and server-generated checklist drafts.

Where it remains weak

- Verification coverage is still thinner than the architecture deserves.
- Subject/component validation needs to extend beyond torque to other measurement families such as volume and pressure.
- Rejection observability is incomplete: the system can fail closed without always making the reason easy to diagnose.
- The real-document evaluation corpus should be substantially larger and more adversarial.
- Some list/search paths still carry more data client-side than ideal for long-term scaling.
- The repair lifecycle is fragmented after the planning/checklist stage; durable repair outcomes and maintenance history are comparatively thin.

Engineering bottlenecks

The primary bottleneck is now validation work. Changes to retrieval, extraction, models, thresholds, safety rules, and planner behavior increasingly require real-corpus tests, eval runs, interpretation, implementation, regression coverage, and browser verification. That is slower than adding UI, but it is the right kind of slowness for a repair-information product.

Product bottleneck

Natural repair lifecycle: symptom -> diagnosis -> evidence -> plan -> work performed -> parts/tools used -> result -> maintenance/history

Corolla Fix Helper handles much of the left half of this lifecycle exceptionally well. The right half is thinner. Closing that gap is the largest product opportunity.

3. SWOT Analysis

Area	Assessment
Strengths	Local storage; simple deployable architecture; strong evidence contract; exact-page source access; real-corpus evaluation culture; hybrid retrieval; mature automated tests; backup/restore; safety-aware planner; usable without AI.
Weaknesses	Small verified evaluation set; imperfect semantic grounding; difficult PDFs remain difficult; some scaling behavior remains client-heavy; repair lifecycle does not yet form a complete history; AI still depends on cloud APIs.
Opportunities	Excellent personal repair history; document quality diagnostics; evidence-aware maintenance records; lightweight DTC/diagnostic capture; local speech-to-text; selective local-model support; richer repair-mode mobile UI.

Threats	Scope creep into garage-management software; replacing simple architecture with fashionable frameworks; trusting multimodal models too much; silent ingestion corruption; model-provider regressions; premature multi-vehicle abstraction; excessive API dependence.
---------	--

Competitive context reinforces this assessment. ALLDATA, Mitchell1, Snap-on, and modern garage platforms are strong because of data breadth, diagnostics, workflow integration, and business operations. Attempting to recreate those moats is strategically poor. Corolla Fix Helper is better positioned as a trusted personal vehicle knowledge system.

4. Major Architectural Findings

A. Keep SQLite

Decision: KEEP. Confidence: Very high. SQLite matches the local-first product model unusually well: portable data, simple backup, no service process, low operational burden, transactions, and easy recovery. Changing databases would create work without corresponding user value.

B. Do not add a vector database yet

Decision: DELAY. Confidence: Very high. Current corpus size and hybrid retrieval remain tractable. A vector database should be justified by measured latency, much larger corpus scale, complex filtered vector queries, or multi-vector representations - not by fashion.

C. Do not migrate Repair Planner to Agents SDK now

Decision: DELAY. Confidence: High. The app already owns the hard part: application-specific evidence, safety, and trust contracts. An SDK migration would replace orchestration plumbing while preserving most custom logic. Revisit if the Planner grows into many tools, resumable workflows, multiple human approvals, or tracing complexity.

D. LangGraph is not justified on the current roadmap

Decision: REJECT. Confidence: Very high. The current planner is bounded and short-lived. Durable graph checkpoints, long-lived interrupts, and workflow time travel solve problems the app does not currently have.

E. Treat the evidence contract as strategic infrastructure

Decision: DOUBLE DOWN. Confidence: Very high. Any future feature that communicates torque, capacities, pressures, dimensions, adjustments, parts, procedural steps, or hazard-related claims should inherit the same verified-evidence discipline.

F. Build an extraction quality pipeline, not a global clever extractor

Decision: ADD SELECTIVELY. Confidence: Very high. The failed reading-order experiment proved that synthetic success can hide real-corpus damage. Future ingestion work should detect, score, flag, selectively repair, compare, and approve rather than reprocess everything.

5. Product Findings

The product's real category

Corolla Fix Helper is becoming a personal vehicle repair knowledge system, not merely a RAG app.

AI is one way the system interrogates the owner's knowledge base. That framing changes priorities: repair history, source quality, extraction reliability, search, and evidence navigation remain valuable even if the model provider changes.

What users of repair products value

Professional repair-information systems consistently emphasize rapid access to specifications, procedures, wiring, diagnostic trouble codes, common fixes, maintenance schedules, and vehicle-specific information. The relevant lesson is not to copy their feature count; it is to reduce the time between 'something is wrong' and 'I know what the source says and what I should do next.'

6. Top Opportunities - Ranked

Rank	Opportunity	Decision	Value	Effort	Maintenance	Confidence
1	Real-corpus evaluation expansion	Add now	Very high	Medium	Low	Very high
2	Evidence-contract generalization and rejection observability	Add now	Very high	Medium	Medium	Very high
3	Document quality diagnostics + selective extraction fallback	Add	Very high	Medium/high	Medium	High
4	Complete repair / maintenance history loop	Add	Very high	Medium	Low/medium	High
5	Search / retrieval tuning based on evals	Improve incrementally	High	Medium	Medium	High
6	Remaining performance / server pagination work	Add	High	Low/medium	Low	High
7	Mobile repair mode	Add	High	Medium	Low	High

8	Lightweight diagnostic / DTC records	Prototype	Medium/high	Medium	Medium	Medium-high
9	Better image assistance	Expand cautiously	Medium	Medium	Medium	Medium-high
10	Local speech-to-text	Prototype later	Medium	Low/medium	Low	Medium-high
11	Local LLM provider	Experiment later	Medium	High	High	Medium
12	Multi-vehicle support	Delay	Eventually high	High	High	High

1. Expand the real-corpus eval suite

This is the highest-return engineering task. Target 25-40 verified repair questions by the end of the first quarter, including positive answers, refusals, ambiguous questions, conflicting sources, wrong-component same-number traps, follow-ups, safety-critical cases, table-derived data, noisy OCR, and difficult layouts. Every retrieval or model change should then become measurable.

2. Finish evidence integrity before expanding AI

Implement the open evidence-hardening items early. Generalize subject validation so a matching number alone cannot establish that it applies to the right component. Add rejection-path metrics and a shared response schema so false negatives are diagnosable.

3. Build a document-quality subsystem

Add quality flags such as `low_text`, `ocr_used`, `suspected_duplicate_runs`, `suspected_table`, `reading_order_risk`, `embedding_missing`, and `page_parse_error`. Create a maintenance view that highlights only the pages/documents requiring attention. Alternative extractors should be evaluated only on problematic pages.

4. Create a first-class repair history

A completed repair should record date, mileage, symptom, plan/checklist, procedure, documents/evidence, parts used, costs, notes/photos, result, and follow-up. Do not add customers, invoicing, labor billing, technician scheduling, or shop-management features.

5. Improve retrieval only when evals prove it

Test changes sequentially: exact phrase and field weighting, RRF tuning, chunk structure, neighbor expansion, reranking, and only then more exotic retrieval. Keep a change only when real evals improve.

6. Finish mundane scalability work

Finish bounded data access across normal record pages, reduce unnecessary payloads, and measure load time using the actual 1,443-document corpus plus larger synthetic corpora. Set budgets for payload size, API latency, DOM count, retrieval latency, memory, and import throughput.

7. Create a mobile repair mode

Do not build a native app. Use the existing PWA and add a hands-busy mode: large text, one checklist step at a time, previous/next controls, quick photo/note capture, safety warning, source-page button, and simple completion controls.

8. Diagnostics: records before hardware integration

Create Diagnostic Sessions containing DTCs, freeze-frame values, selected PIDs, symptoms, date/mileage, screenshots/photos, notes, and related documents/procedures. Allow manual entry or paste/import first. Add live adapter integration only after the information model proves valuable.

9. Image understanding: observe, do not certify

Use images to identify visible components, extract labels or part numbers, describe leak locations, compare saved photos, or assist search. Continue requiring technical specifications and safety-critical conclusions to come from verified document evidence.

10. Voice: local transcription first

A full conversational voice agent is unnecessary. A useful first step is microphone -> local transcription -> Ask input / note / checklist note. This improves hands-busy use without complicating the core architecture.

11. Local LLMs: build an adapter, not a commitment

Introduce a provider abstraction only after evaluation work is strong enough to compare models fairly. The evidence contract should remain above both cloud and local model providers. Do not make local inference the default unless it passes the same quality and refusal gates.

7. Features to Avoid

General-purpose chatbot behavior: It weakens the defining source-grounding constraint.

Shop management: Estimates, customers, invoices, payments, technician scheduling, multi-shop reporting, and CRM are a different product category.

Social/community repair content: Moderation and trust become an entire new product problem.

Automatic internet repair advice mixed with local evidence: External findings should never be blended invisibly with owner-document evidence.

Autonomous long-running agents: Do not create chains simply because frameworks make them possible.

Vector database before measured need: Avoid adding infrastructure without a performance justification.

Native mobile apps before the PWA hits a real limitation: Current web/PWA architecture is sufficient for the next phase.

Full live scanner integration before diagnostic records prove value: Start with information capture, not hardware complexity.

Computer-vision diagnosis marketed as authoritative: Use vision as observational assistance, not a safety certification layer.

8. Risks and Mitigations

Risk	Severity	Mitigation
Correct number, wrong component	High	Generalize subject validation and add adversarial evals.
False evidence rejection	High	Add rejection telemetry and rejection-focused eval cases.
PDF extraction silently corrupts meaning	High	Page quality scoring, real-corpus diffing, selective fallback only.

Model update changes behavior	High	Pinned model versions plus mandatory eval gate before upgrades.
Feature explosion increases maintenance	High	Require an opportunity score and architecture-cost review for major features.
Client-side scaling regresses	Medium/high	Performance budgets plus 1.4k/5k/10k fixtures.
OpenAI/network unavailable	Medium	Preserve non-AI operation; evaluate local provider later.
Local model gives weaker safety behavior	High	Use the same eval suite and evidence contract; never bypass validation.
Repair history becomes shop software	Medium	Keep explicit single-owner/single-vehicle workflow scope.
Multi-vehicle creates pervasive complexity	Medium/high	Wait until single-vehicle domain and lookup seams are stable.

9. Recommended 12-Month Roadmap

Next 30 days - Trust and measurement

- Finish open evidence-hardening and rejection-observability issues.
- Expand the verified real-manual eval corpus.
- Add conflicting-evidence and same-number/wrong-component cases.
- Run and record a complete quality baseline: latency, retrieval accuracy, answer quality, refusals, and cost.
- Finish obvious server-pagination/performance debt and define budgets.
- Add a document-health report before attempting another extraction algorithm.

Exit condition: you can change retrieval, extraction, or the answer model and quantitatively say whether the app became better or worse. *Confidence: Very high.*

Next quarter - Close the repair lifecycle

- Build Repair History / Maintenance Records.
- Link completed checklists to symptoms, procedures, documents, photos, and notes.
- Track mileage, date, parts, cost, result, and follow-up.
- Build the first mobile Repair Mode.
- Add document-quality flags and trial one structure-aware extraction fallback only on known-bad pages.
- Tune retrieval only where expanded evals identify failures.

Exit condition: the app not only answers 'how do I fix this?' but remembers 'what did I actually do, when, with what result?' *Confidence: High.*

Next 6 months - Make diagnosis more useful

- Introduce Diagnostic Sessions and manual DTC/PID capture.
- Make DTCs searchable against the local document corpus.
- Link symptoms to diagnostic sessions.
- Extend image assistance for observation/search while preserving document-sourced specs.
- Prototype local speech-to-text for Ask, notes, and checklists.
- Continue mobile repair-mode refinement from real use.
- Run a local-model feasibility evaluation behind an experimental provider interface.

Exit condition: observed vehicle behavior, diagnostic evidence, repair manuals, and repair history are connected without turning the app into a scanner platform. *Confidence: Medium-high.*

Next 12 months - Selective expansion

- Choose one major expansion based on usage and evidence.
- Preferred order: local/offline answer model; multi-vehicle support; limited live diagnostic adapter integration.
- Do not attempt all three.
- Reconsider Agents SDK only if the Planner has accumulated many tools, approvals, resumable workflows, or tracing needs.

Exit condition: one evidence-backed expansion, not three partially built platforms. *Confidence: High.*

10. Explicitly Not Recommended

OpenAI Agents SDK migration now

The app already owns the hard part - evidence, trust, and safety contracts. An SDK migration would mostly replace working orchestration infrastructure. Revisit in 6-12 months if orchestration complexity materially grows.

LangGraph migration

Current workflows do not justify checkpoint graphs, durable interrupts, and time-travel state.

Vector database

No current performance evidence supports it. Revisit only when corpus scale or measured retrieval latency exceeds a defined budget.

Full offline AI as a headline feature

Offline operation is not a win if answer quality drops materially. Quality parity comes first.

Cloud sync / accounts

This changes identity, authorization, encryption, backups, conflict resolution, hosting, privacy, and operating cost; it conflicts with the current architecture unless multi-device usage becomes a demonstrated need.

Multi-vehicle immediately

It affects vehicle identity, record ownership, filtering, URLs, prompts, retrieval, backups, imports, diagnostics, and relationships. Delay until the single-vehicle domain is finished.

Parts inventory management

Simple parts-needed/parts-used records are useful; stock counts, suppliers, purchase orders, and inventory management are not.

Automatic web repair research in the main evidence pipeline

Owner documents and external web claims have different provenance and trust levels and should remain separate.

Autonomous repair instructions from images

Not appropriate for a safety-oriented product.

11. Open Questions Requiring Further Research

- What number of verified real-document questions is enough for the eval suite to become representative? Start with 25-40, but let failure coverage determine the final size.
- Which extraction defects actually harm retrieval or answer quality? Quantify the effect before investing heavily in extraction repair.
- Can a structure-aware extractor improve only known-bad pages without harming ordinary PDFs?
- Can a local model meet the current answer/refusal/evidence gate on the available hardware at acceptable latency?
- Would live diagnostic integration materially outperform manual DTC/PID entry for this product?
- When does another vehicle become a real user need rather than an architectural curiosity?

12. Final Recommendation

Improve the quality of the repair decision before increasing the number of things the app can do.

The roadmap should move in this order:

Trust -> source quality -> retrieval -> repair workflow/history -> mobile usability -> diagnostics -> selective offline AI -> broader scope

The architecture is currently simpler than the cutting edge, but that is mostly a strength. The project already has enough retrieval sophistication, enough AI orchestration, and enough infrastructure to support its next phase. The highest return now comes from stronger evidence, a better corpus, better evaluations, complete repair history, and real-world usability.

Most importantly, the project has already demonstrated that it is willing to reject technically impressive changes when real evidence says they make the product worse. That is exactly the decision-making behavior that should govern the next twelve months.

Overall recommendation confidence: Very high.

Source Notes

Internal evidence reviewed included the current Corolla Fix Helper README, roadmap, architecture and quality-testing documentation; open evidence-integrity issues; recent merged pull requests covering Ask/RAG hardening, Repair Planner safety and evidence contracts, search pagination, URL state, layout fixes, accessibility, and copy cleanup; and real-corpus evaluation findings.

External comparison considered current public documentation and product positioning for ALLDATA, Mitchell1, Snap-on Diagnostics, Tekmetric, OpenAI Agents SDK, LangGraph, Qdrant hybrid retrieval, Tesseract/PaddleOCR, llama.cpp, and whisper.cpp.